RL-Driven Request Batching Reverse Proxy

Replacing Static Timers with a PPO Reinforcement Learning Agent

R Abinav (ME23B1004)
Sudarshan S (CS23B2007)
Chris Jason (CS23B1012)
Shirish Giroti (CS23B2041)
Ashrith Yathin (CS23B2006)

https://github.com/Raifu-Sutairu/req-batching

Course: Reinforcement Learning (CS3009)

The Problem: Every Request Hits the Upstream

Without Batching

- Each client request triggers one upstream call independently
- Under bursty traffic, N simultaneous requests = N upstream calls
- Upstream bears full load with no coalescing
- Static workarounds (fixed timer, size cap) are blind to traffic state
- gzip middleware silently invalidates `ETag` headers, forcing every conditional request (`If-None-Match`) to hit upstream with no cache benefit.

With Intelligent Batching

- Concurrent requests to the same endpoint are parked in a shared slot
- One upstream call serves N waiting clients
- Flush timing adapts to queue depth, arrival rate, and backend health
- The question is not whether to batch - it is *when* to flush

Related Work & Inspiration

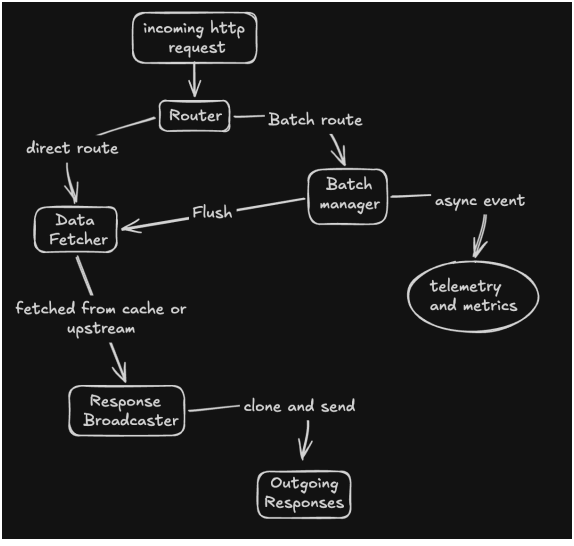
- **1. Cloudflare "Sometimes I Cache" (2024)**
  Exponential urgency formula $p(t) = e^{-\lambda \cdot \text{remaining}}$ is optimal for age only decisions. We extend this to a 4 signal observation space conditioning on queue depth and arrival rate.

- **2. Cold RL, Bhayani, arXiv:2508.12485 (2025)**
  Offline RL for NGINX cache eviction via DQN + ONNX sidecar + 500μs hard timeout + LRU fallback. Directly validates our offline first training, ONNX serving, and hard deadline fallback pattern.

- **3. IEEE 10740859**
  Entropy regularisation in PPO prevents policy collapse to always WAIT or always FLUSH. Directly motivated our exploration strategy ($\beta = 0.01$ entropy coefficient).

- **4. Sarathi Serve, Agrawal et al., OSDI 2024**
  Same throughput and latency tradeoff in LLM inference batching. Different domain (GPU inference vs HTTP proxy), same core tension, validates the problem framing.

Reverse Proxy Architecture



Our Solution: A Rust Reverse Proxy with RL Flush Control

Request Flow

- TCP connection accepted via Tokio listener
- Semaphore caps max concurrent connections
- Router fingerprints request: GET -> batch, non-GET -> pass-through

Batching Engine

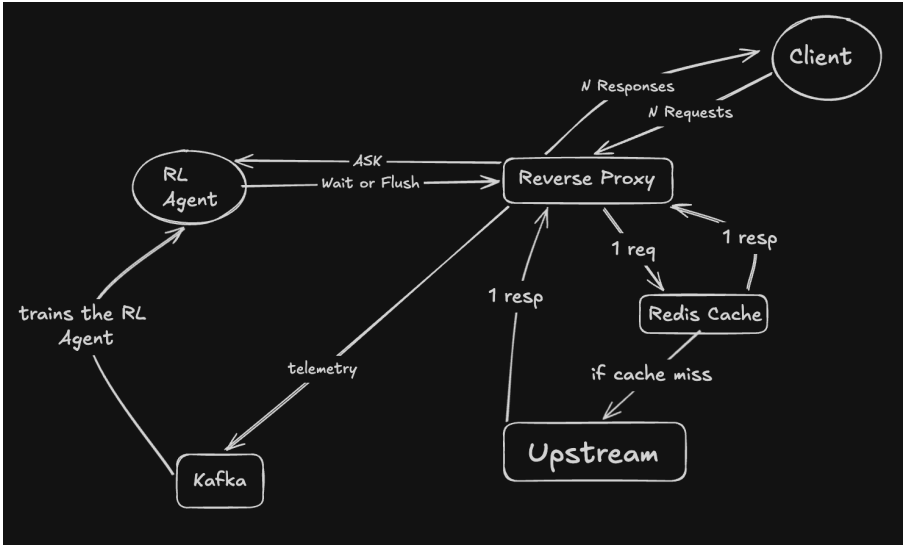
- DashMap registry groups requests by endpoint key
- Incoming request parks via `oneshot` channel
- Timer task spawned once per BatchSlot on creation

Flush Decision

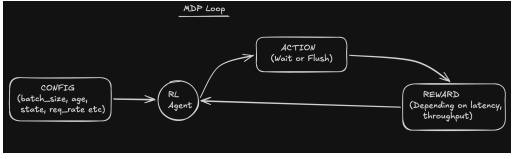
- Hard limits checked unconditionally first
- RL Agent queried only inside safe envelope
- Fan-out: `mem::take` drains senders, all clients receive response simultaneously

Written in Rust - Tokio + Hyper + DashMap. Zero lock contention across concurrent endpoints.

System Architecture



RL Algorithm: MDP Formulation



Component	Definition
State s	$[\text{batch\_size}/128,\ \text{age\_ms}/50,$ $\log_{1p}(\text{p99})/\log_{1p}(500),$ $\tanh(\text{rate}/100)]$
Action a	$\{0 = \text{WAIT},\ 1 = \text{FLUSH}\}$ binary decision per incoming request
Reward r	$\log_2(1 + \text{sz}) + \text{urgency}$ $-\beta \cdot \text{upstream\_penalty}$ $-\gamma \cdot \mathbb{1}[\text{forced}]$

- Episode = one BatchSlot lifetime, from first request arrival to flush.
- Agent is queried on every request *within* the safety envelope - hard limits fire unconditionally first.

RL Algorithm: Why PPO with Offline Pre-training?

Algorithm Selection

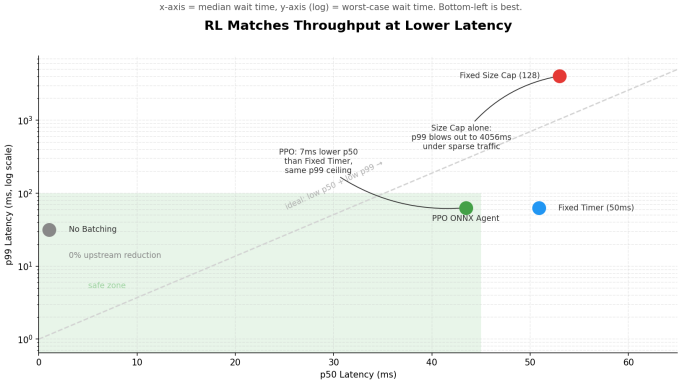
Not DQN	Experience replay not suited to per-request sub-5ms decision frequency
Not plain PPO	On-policy only; wastes heuristic telemetry from before agent exists
PPO + Offline ✓	Warm-start from Kafka replays; entropy bonus $\beta = 0.01$ prevents degenerate policy collapse

Network Architecture

- `Input: obs [4]`
- `Linear(4 -> 64) + LayerNorm + ReLU`
- `Linear(64 -> 64) + LayerNorm + ReLU`
- `Linear(64 -> 2): WAIT or FLUSH logits`
- `Categorical distribution -> action`

- Separate actor and critic networks - no shared layers (non-stationary reward scale).
- Trained offline on Kafka `batch-telemetry`, exported to ONNX, served via gRPC sidecar at <1ms inference.

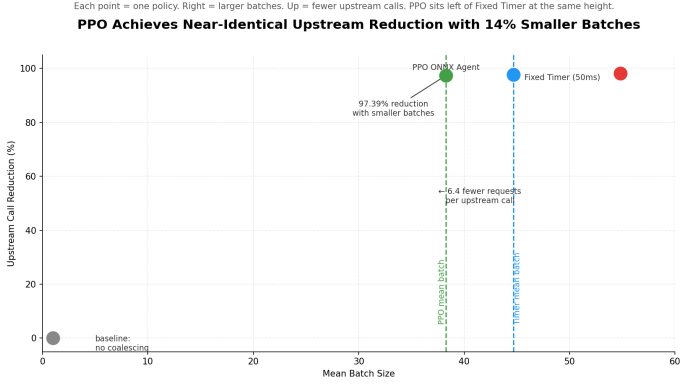
Results: Latency Profile Across Policies



Key:

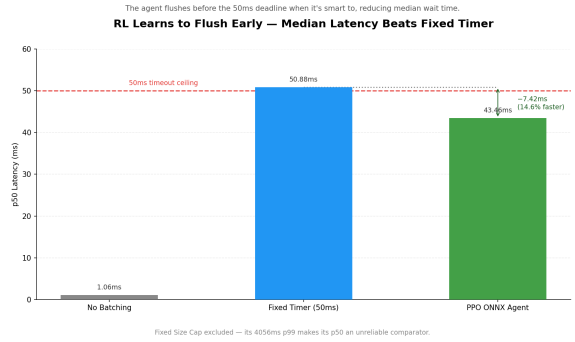
- PPO: lower p50 (43ms vs 51ms) with identical p99 ceiling of 63ms. Hard timeout enforces the ceiling unconditionally.
- Fixed Size Cap without a timer: p99 blows to 4056ms under sparse traffic. This is the exact failure mode the safety envelope prevents.
- Bottom-left is best. PPO is closer to that corner than any batching policy.

Results: Upstream Reduction vs Mean Batch Size

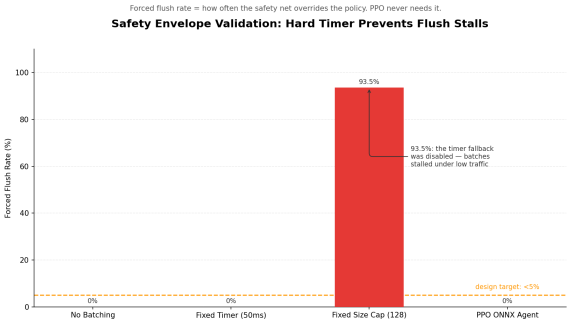


- PPO: 97.39% upstream reduction with 14% smaller mean batches than Fixed Timer. Flushes early when justified, not when the clock runs out.
- Cloudflare age-only policy achieves 82.16%. The gap is explained by conditioning on `request_rate` and `upstream_p99`, signals unavailable to a single-signal policy.
- High reduction and small batches together mean less upstream load and lower median latency simultaneously.

Results: Median Latency & Safety Envelope Validation



Agent flushes early when smart to - 14.6% lower median latency than fixed 50ms timer.



PPO: 0% forced flush rate. Fixed Size Cap: 93.5% - stalls under sparse traffic without a timer fallback.